

MantenIA

Manual técnico

Backend · Mantenia.Api

Cómo está implementado, paso a paso, y cómo funciona en detalle:
arquitectura, base de datos, seguridad, endpoints y lógica de negocio.

.NET 10 · ASP.NET Core · Entity Framework Core · SQL Server · JWT

Septiembre 2026

Contenido

1. Qué es el backend y cómo está pensado	4
1.1 Arquitectura en capas	4
1.2 Tecnologías utilizadas	4
2. Estructura de carpetas	6
3. Arranque de la aplicación (Program.cs)	7
3.1 Inyección de dependencias	7
4. Recorrido completo de un pedido	8
5. Acceso a datos con Entity Framework Core	9
5.1 Base existente, sin migraciones	9
5.2 Entidades	9
5.3 Configuraciones (Fluent API)	9
5.4 Modelo de datos usado	9
5.5 Cómo se consulta	10
6. Seguridad	11
6.1 Claves: nunca en texto plano	11
6.2 Login y token JWT	11
6.3 Permisos por rol	11
6.4 Aislamiento por organización (multi-tenant)	12
7. Endpoints de la API	14
7.1 Autenticación — AuthController	14
7.2 Pantallas de la app — ApplicationController	14
7.3 Configuración — AjustesController y SistemasController	14
8. Lógica de negocio detalle por detalle	16
8.1 Indicadores: cómo se “predicen” las fallas	16
8.2 La “foto” de la organización (Snapshot)	16
8.3 Dashboard (pantalla de inicio)	17
8.4 Reportes	17
8.5 Órdenes de trabajo	17
8.6 Lectura del código QR	17
8.7 Ajustes y Sistemas: reglas comunes	18
9. DTOs, validaciones y manejo de errores	19
9.1 ¿Qué es un DTO y por qué no devolver las entidades?	19
9.2 Validación automática	19
9.3 Errores: siempre la misma forma	19

10. Configuración y puesta en marcha	21
10.1 Cómo levantarla	21
10.2 appsettings	21
10.3 Scripts SQL	21
10.4 Crear un usuario	21
11. Preguntas frecuentes para la defensa	22
12. Glosario	23

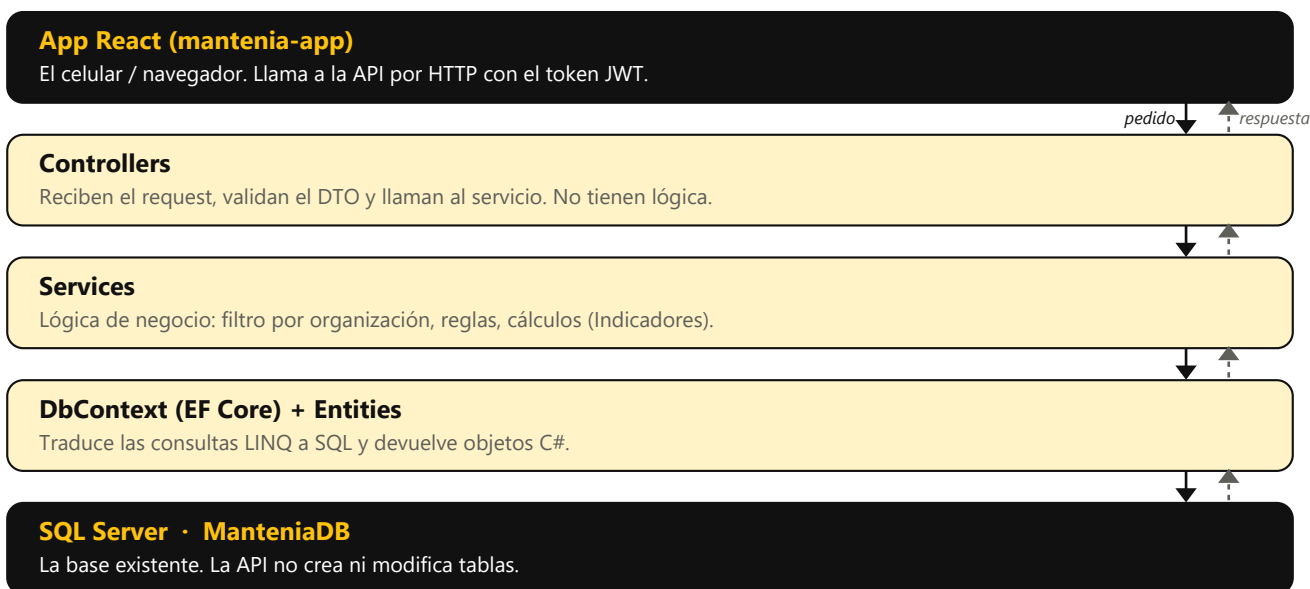
1. Qué es el backend y cómo está pensado

MantenIA es una app de mantenimiento: permite registrar activos (máquinas, vehículos, equipos), cargar órdenes de trabajo con sus repuestos y anticipar fallas a partir del historial. El **backend** (la carpeta `Mantenia.Api`) es el programa que corre en el servidor: recibe los pedidos de la app, aplica las reglas de negocio y lee o escribe en la base de datos SQL Server `ManteniaDB`.

La app (el *frontend*, en `mantenia-app`) nunca habla con la base directamente: todo pasa por la API. Eso permite controlar en un solo lugar **quién** puede ver y hacer **qué**.

1.1 Arquitectura en capas

El backend está organizado en capas. Cada capa tiene una sola responsabilidad y solo conoce a la de abajo:



Capa	Responsabilidad	Qué NO hace
Controller	Define la ruta (URL), recibe el JSON, lo valida y devuelve el código HTTP correcto (200, 404...).	No calcula ni consulta la base.
Service	Aplica las reglas: filtra por organización, valida referencias, calcula indicadores, arma la respuesta.	No sabe de HTTP (no devuelve códigos de estado).
DbContext / Entities	Representa las tablas como clases C# y traduce las consultas a SQL.	No tiene reglas de negocio.

Para defenderlo

“Separamos en capas para que cada parte tenga una única responsabilidad. Si mañana cambia la base, se toca la capa de datos; si cambia una regla, se toca el servicio; si cambia la URL, el controller. Así el código es más fácil de entender, probar y mantener.”

1.2 Tecnologías utilizadas

Tecnología	Para qué se usa
.NET 10 / ASP.NET Core	Framework web de Microsoft con el que está hecha la API (lenguaje C#).
Entity Framework Core 10	ORM: permite consultar la base con código C# (LINQ) en vez de escribir SQL a mano.
SQL Server	Motor de base de datos donde vive ManteniaDB.
JWT (JSON Web Token)	Autenticación: después del login, cada pedido lleva un token firmado que identifica al usuario.
ASP.NET Core Identity – PasswordHasher	Guarda las claves con hash PBKDF2 (nunca en texto plano).
OpenAPI + Scalar	Documentación automática de los endpoints y una interfaz web para probarlos (/scalar/v1).

Paquetes NuGet del proyecto (Mantenía.Api.csproj):

Microsoft.AspNetCore.Authentication.JwtBearer, Microsoft.AspNetCore.OpenApi, Microsoft.EntityFrameworkCore.SqlServer y Scalar.AspNetCore. No hay más dependencias.

2. Estructura de carpetas

Cada carpeta agrupa un tipo de pieza. Son 74 archivos C#: solo lo que usa la app.

Carpeta / archivo	Qué contiene
Program.cs	Punto de entrada: arma la aplicación, registra servicios y define el orden de los middlewares.
Controllers/	4 controllers: AuthController (login), AppController (pantallas de la app), AjustesController (catálogos de la organización) y SistemasController (configuración general).
DTOs/	Clases que definen <i>qué JSON entra y qué JSON sale</i> de cada endpoint. <code>AuthDtos.cs</code> y la subcarpeta <code>App/</code> .
Services/	<code>AuthService</code> (login, alta, cambio de clave), <code>TokenService</code> (genera el JWT) y <code>PasswordHasherService</code> (hash de claves).
Services/App/	La lógica de cada pantalla (Activos, Órdenes, Dashboard, Contexto, Ajustes, Sistemas) más las piezas transversales: DatosOrganizacion (filtro por organización), Indicadores (MTBF, riesgo, km), Permisos (roles) y CurrentUser (usuario del request).
Interfaces/	Contratos (interfaces) de los servicios. Los controllers dependen de la interfaz, no de la clase concreta.
Entities/	Una clase por tabla usada (Activo, OrdenTrabajo, Cliente...), con sus columnas y navegaciones (relaciones).
Data/	ManteniaDbContext (la conexión con la base) y <code>Configurations/</code> : cómo se mapea cada tabla (nombre, largo de columnas, claves foráneas).
Middleware/	GlobalExceptionHandler : convierte cualquier error en una respuesta JSON prolija (<code>ProblemDetails</code>).
Extensions/	ServiceCollectionExtensions : registro de dependencias, configuración del JWT y de las políticas de autorización.
Configuration/	Clases que leen secciones de <code>appsettings.json</code> (Jwt y Auth).
Common/	ConflictException : excepción para reglas de negocio violadas (se responde 409).
OpenApi/	Agrega el esquema "Bearer" a la documentación para poder probar con token desde Scalar.
Scripts/	SQL de datos iniciales: rol Sistemas y estado de activo "Programado".

3. Arranque de la aplicación (Program.cs)

Cuando se ejecuta `dotnet run`, lo primero que corre es `Program.cs`. Tiene dos etapas: **registrar servicios** (qué piezas existen) y **armar el pipeline** (por qué pasos atraviesa cada pedido).

Mantenia.Api/Program.cs (resumido)

```
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddPersistencia(builder.Configuration) // DbContext con la cadena de conexión
    .AddSeguridadJwt(builder.Configuration) // JWT, políticas por rol, hash de claves
    .AddAplicacion();                       // servicios de pantallas, controllers, errores

builder.Services.AddCors(...);             // qué frontends pueden llamar a la API

var app = builder.Build();

app.UseExceptionHandler();                // 1. atrapa cualquier error
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();                       // /openapi/v1.json
    app.MapScalarApiReference();            // /scalar/v1 (probar endpoints)
}
app.UseHttpsRedirection();                // 2. fuerza HTTPS
app.UseCors();                            // 3. permite el origen de la app
app.UseAuthentication();                  // 4. lee y valida el token → sabe QUIÉN es
app.UseAuthorization();                   // 5. verifica permisos → sabe si PUEDE
app.MapControllers();                     // 6. deriva al método del controller
app.Run();
```

3.1 Inyección de dependencias

En vez de que cada clase cree sus propias dependencias (`new AuthService(...)`), ASP.NET Core las **inyecta** por el constructor. Se registran una sola vez en `ServiceCollectionExtensions.cs`:

Extensions/ServiceCollectionExtensions.cs

```
services.AddScoped<ICurrentUser, CurrentUser>();
services.AddScoped<DatosOrganizacion>();
services.AddScoped<IActivosAppService, ActivosAppService>();
services.AddScoped<IOrdenesAppService, OrdenesAppService>();
...
services.AddSingleton<ITokenService, TokenService>();
```

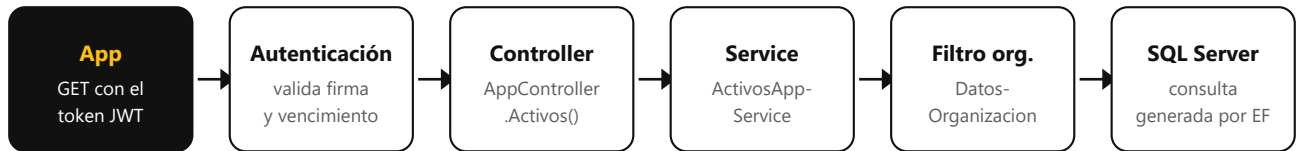
Ciclo de vida	Significado	Se usa para
Scoped	Una instancia nueva por cada request HTTP.	Servicios que usan la base (DbContext) y el usuario actual: cada pedido tiene los suyos.
Singleton	Una sola instancia para toda la aplicación.	Piezas sin estado: TokenService, PasswordHasherService.

Para defenderlo

"Usamos inyección de dependencias porque desacopla las clases: el controller pide una *interfaz* (`IActivosAppService`) y el framework le da la implementación. Facilita reemplazar piezas y hacer pruebas."

4. Recorrido completo de un pedido

Veamos paso a paso qué pasa cuando la app abre la lista de activos: GET /api/app/activos?filtro=riesgo.



1. **La app envía el pedido** con el header `Authorization: Bearer <token>`.
2. **UseAuthentication** valida el token: que la firma sea correcta (clave secreta del servidor), que no esté vencido y que el emisor/audiencia coincidan. Si falla → **401**. Si está bien, arma el User con los datos del token (el id del usuario).
3. **UseAuthorization** revisa el atributo `[Authorize]` del controller. En Ajustes y Sistemas además chequea el rol (capítulo 6). Si no tiene permiso → **403**.
4. **El controller** (`AppController.Activos`) recibe los parámetros `buscar` y `filtro` y llama al servicio. No hace nada más.
5. **El servicio** pide a `DatosOrganizacion` los activos y órdenes *de la organización del usuario* (una “foto” o *Snapshot*), calcula la salud de cada activo con `Indicadores`, filtra por riesgo y ordena.
6. **EF Core** traduce la consulta LINQ a SQL, la ejecuta en SQL Server y devuelve objetos C#.
7. **La respuesta** se arma como DTO (`ActivosListadoDto`) y ASP.NET la serializa a JSON en camelCase. La app recibe un **200** con la lista.

Controllers/AppController.cs – un controller “delgado”: solo delega

```
[HttpGet("activos")]
public async Task<ActionResult<ActivosListadoDto>> Activos(
    [FromQuery] string? buscar, [FromQuery] string? filtro, CancellationToken ct)
    => Ok(await activos.ListarAsync(buscar, filtro, ct));
```

Dato

El `CancellationToken ct` que viaja por todos los métodos permite cancelar la consulta a la base si el usuario cierra la pantalla antes de que termine: no se gastan recursos en una respuesta que nadie va a leer.

5. Acceso a datos con Entity Framework Core

5.1 Base existente, sin migraciones

La base ManteniaDB ya existía. Por eso el backend usa el enfoque **Database First**: las clases se escribieron para calzar con las tablas existentes y **la API no crea ni modifica tablas** (no usa migraciones). Los datos iniciales que hicieron falta se cargan con scripts SQL en Scripts/.

5.2 Entidades

Cada tabla usada tiene una clase en Entities/. Las propiedades son las columnas; las propiedades de tipo objeto son **navegaciones** (relaciones por clave foránea) que permiten ir de una tabla a otra:

Entities/Activo.cs (resumido)

```
public class Activo
{
    public int IdActivo { get; set; }
    public int? IdCategoriaActivo { get; set; } // FK → CategoriaActivo
    public int? IdCliente { get; set; } // FK → Cliente
    public int? IdEstadoActivo { get; set; } // FK → EstadoActivo
    public string? Identificador { get; set; } // lo que va en el QR
    public string? Nombre { get; set; }
    ...
    public CategoriaActivo? CategoriaActivo { get; set; } // navegación
    public Cliente? Cliente { get; set; }
}
```

5.3 Configuraciones (Fluent API)

Cómo se mapea cada entidad está en Data/Configurations/: nombre de tabla, clave primaria, largo de columnas y relaciones. El DbContext las carga todas automáticamente:

Data/Configurations/ActivoConfiguration.cs (resumido)

```
builder.ToTable("Activo", "dbo");
builder.HasKey(e => e.IdActivo);
builder.Property(e => e.Nombre).HasMaxLength(150).IsUnicode(false); // varchar(150)
builder.HasOne(e => e.Cliente).WithMany()
    .HasForeignKey(e => e.IdCliente)
    .OnDelete(DeleteBehavior.Restrict); // no borra en cascada
```

5.4 Modelo de datos usado

Grupo	Tablas	Para qué
Organización	Organizacion, TipoOrganizacion, Suscripcion, PlanSaaS	Quién es el cliente de Mantenia, de qué rubro es y qué plan tiene.
Usuarios	Usuario, Rol	Quién entra a la app y con qué permisos.
Activos	Activo, CategoriaActivo, EstadoActivo, Cliente, Ubicacion, Localidad	Lo que se mantiene, a quién pertenece y en qué estado está.
Órdenes	OrdenTrabajo, OrdenTrabajoDetalle, TipoTrabajo, EstadoOrden, Prioridad, TipoTrabajoEstadoOrden	Los trabajos realizados o por hacer, sus repuestos y su flujo de estados.

Grupo	Tablas	Para qué
Personalización	Modulo, TipoOrganizacionModulo, LabelTipoOrganizacionModulo	Qué módulos ve cada rubro y cómo se llaman las cosas (ej. "Vehículos" en un taller).

5.5 Cómo se consulta

Services/App/DatosOrganizacion.cs

```
var ordenes = await db.OrdenesTrabajo.AsNoTracking()
    .Where(o => idsVisibles.Contains(o.IdActivo)) // → WHERE ... IN (subconsulta)
    .Include(o => o.TipoTrabajo)                // → JOIN TipoTrabajo
    .Include(o => o.EstadoOrden)                 // → JOIN EstadoOrden
    .ToListAsync(ct);                           // ejecuta el SQL
```

- **AsNoTracking()**: para lecturas. EF no guarda copia para detectar cambios → más rápido y menos memoria.
- **Include()**: trae los datos relacionados en la misma consulta (JOIN), evitando una consulta por fila.
- Para escribir: se agrega o modifica la entidad y se llama a `SaveChangesAsync()`, que genera el INSERT/UPDATE/DELETE.

Para defenderlo

"Usamos un ORM porque las consultas se escriben en C# con chequeo de tipos: si una columna cambia de nombre, el proyecto no compila en vez de fallar en producción. Además EF parametriza las consultas, lo que evita inyección SQL."

6. Seguridad

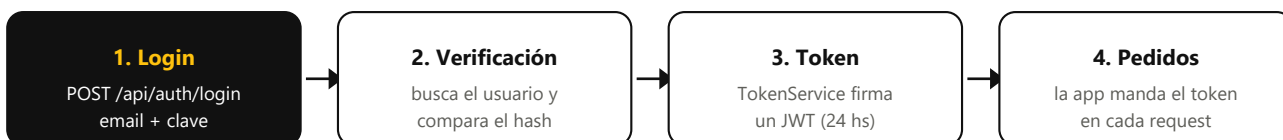
La seguridad tiene cuatro piezas que se complementan: **claves hasheadas**, **token JWT**, **permisos por rol** y **aislamiento por organización**.

6.1 Claves: nunca en texto plano

Al crear un usuario, la clave se transforma con **PBKDF2 + HMAC-SHA512**, con una *sal* aleatoria y 100.000 iteraciones (el PasswordHasher de ASP.NET Core Identity). En la columna `ClaveHash` se guarda solo el resultado. Al hacer login se aplica el mismo cálculo a la clave ingresada y se comparan los resultados.

- Si alguien roba la base, no puede leer las claves: el hash no se puede “deshacer”.
- La *sal* hace que dos usuarios con la misma clave tengan hashes distintos.
- Las 100.000 iteraciones hacen muy lento probar millones de claves por fuerza bruta.

6.2 Login y token JWT



El token es un texto con tres partes (encabezado.datos.firma). Los **datos** (claims) que lleva son:

Claim	Contenido
sub	Id del usuario. Es lo único que la API usa del token para identificar a la persona.
email, name	Informativos.
role, id_organizacion	Informativos. Los permisos y la organización se leen de la base en cada request (ver 6.3 y 6.4).
jti, exp, iss, aud	Id único del token, vencimiento (24 hs), emisor y audiencia.

La **firma** se calcula con HMAC-SHA256 y una clave secreta que solo conoce el servidor (`Jwt:Key`, mínimo 32 caracteres; la API no arranca si falta). Si alguien modifica un dato del token, la firma deja de coincidir y la API lo rechaza con 401.

6.3 Permisos por rol

Hay dos roles con permisos especiales, definidos en `Services/App/Permisos.cs`:

Rol	Puede	Endpoints
Administrador	Configurar los catálogos de su organización: categorías, estados y tipos de trabajo.	<code>/api/app/ajustes/...</code>
Sistemas	Todo lo anterior + la configuración general: tipos de organización, módulos, textos, estados de orden, prioridades, estados generales.	<code>/api/app/sistemas/...</code>
Otros (Técnico, Operario)	Usar la app normalmente (activos, órdenes, reportes).	<code>/api/app/...</code> (resto)

Se implementa con **políticas de autorización** de ASP.NET Core. El controller declara qué política exige:

```
[Authorize(Policy = Politicas.Sistemas)]           // SistemasController
[Authorize(Policy = Politicas.AjustesOrganizacion)] // AjustesController

// Registro de las políticas (ServiceCollectionExtensions.cs)
options.AddPolicy(Politicas.Sistemas, p => p
    .RequireAuthenticatedUser()
    .AddRequirements(new RolRequerido(Roles.Sistemas)));

options.AddPolicy(Politicas.AjustesOrganizacion, p => p
    .RequireAuthenticatedUser()
    .AddRequirements(new RolRequerido(Roles.Administrador, Roles.Sistemas)));
```

El RolRequeridoHandler es el que decide: **lee el rol del usuario desde la base** y lo compara con los roles permitidos.

Services/App/Permisos.cs – RolRequeridoHandler

```
var rol = await db.Usuarios.AsNoTracking()
    .Where(u => u.IdUsuario == idUsuario && u.Activo != false)
    .Select(u => u.Rol!.Descripcion)
    .FirstOrDefaultAsync();

if (rol is not null
    && requirement.Roles.Any(r => string.Equals(r, rol.Trim(), OrdinalIgnoreCase)))
{
    context.Succeed(requirement); // si no se llama a Succeed → 403 Forbidden
}
```

Para defenderlo

“El rol no se toma del token sino de la base, en cada pedido. Si a alguien le sacamos el rol de administrador o desactivamos su usuario, pierde el acceso en el momento, sin esperar a que venza el token de 24 horas.”

6.4 Aislamiento por organización (multi-tenant)

Varias organizaciones (un taller, una planta, un servicio técnico) usan la misma base. La regla de oro es que **cada una solo ve y modifica lo suyo**. Se resuelve en dos piezas:

CurrentUser: ¿quién pide y de qué organización es?

Toma el id del usuario del token y **lee su organización de la base** (una vez por request). Así, si un usuario cambia de organización, el cambio aplica en el acto.

DatosOrganizacion: el filtro único

La tabla Activo no tiene columna de organización: un activo pertenece a una organización **por su categoría o por su cliente**. Todas las consultas de activos y órdenes pasan por este único método:

Services/App/DatosOrganizacion.cs

```
public IQueryable<Activo> ActivosQuery()
{
    var org = usuario.IdOrganizacion;
    return db.Activos.AsNoTracking().Where(a =>
        org != null
        && ((a.Cliente != null && a.Cliente.IdOrganizacion == org)
            || (a.CategoriaActivo != null && a.CategoriaActivo.IdOrganizacion == org)));
}
```

Situación	Qué hace la API
Pedir la ficha de un activo de otra organización	Responde 404 , como si no existiera (no revela que existe).
Crear/editar un activo con categoría o cliente de otra organización	Responde 409 : "La categoría elegida no existe."
Crear un activo sin categoría ni cliente	Responde 409 : quedaría sin dueño y nadie lo vería.
Modificar una orden o repuesto de otra organización	Responde 404 : se verifica que el activo de la orden sea visible.
Ajustes: tocar una categoría/estado/tipo de otra organización	Responde 404 : todo se busca filtrando por la organización del usuario.
Usuario sin organización	No ve ningún activo ni puede crear activos o clientes.

Para defenderlo

"El filtro por organización está centralizado en un solo método (*ActivosQuery*). Ningún servicio consulta activos u órdenes por su cuenta: todos parten de ahí. Eso hace imposible olvidarse del filtro en un endpoint nuevo."

"La organización se lee de la base y no del token, por la misma razón que el rol."

7. Endpoints de la API

Todos requieren token, salvo login y registro. Las respuestas son JSON.

7.1 Autenticación — AuthController

Método y ruta	Qué hace
POST /api/auth/login	Recibe email y clave. Devuelve { token, expiraUtc } o 401.
POST /api/auth/registro	Crea un usuario con la clave hasheada. Solo si Auth:PermitirRegistro = true (en producción conviene apagarlo).
POST /api/auth/cambiar-clave	Cambia la clave del usuario logueado; exige la actual.

7.2 Pantallas de la app — AppController

Método y ruta	Pantalla / uso
GET /api/app/contexto	Al iniciar: usuario, organización, perfil (taller/planta), plan, textos y permisos.
GET /api/app/catalogos	Listas de los formularios: categorías, estados, clientes, tipos de trabajo, prioridades...
GET /api/app/dashboard	Inicio: resumen, alerta predictiva, próximas órdenes y trabajos en curso.
GET /api/app/reportes?meses=6	Reportes: serie mensual, costos, activos críticos, clientes a contactar.
GET /api/app/activos	Lista de activos con su salud. Filtros: buscar, riesgo, revisar, detenidos.
GET /api/app/activos/{id}	Ficha del activo con su historial de órdenes.
GET /api/app/activos/codigo/{codigo}	Lectura del QR: busca por Identificador.
GET /api/app/activos/{id}/edicion	Datos para el formulario de edición.
POST • PUT • DELETE /api/app/activos	Alta, edición y baja de activos.
POST /api/app/clientes	Alta rápida de cliente desde el formulario del activo.
GET /api/app/ordenes	Órdenes (abiertas, cerradas o todas; opcionalmente de un activo).
GET /api/app/ordenes/{id}	Detalle con repuestos, costos y estados posibles.
POST /api/app/ordenes	Crea una orden con sus repuestos a nombre del usuario.
PUT /api/app/ordenes/{id}/estado	Cambia el estado y completa fechas de inicio/cierre.
POST /api/app/ordenes/{id}/repuestos	Agrega un repuesto a la orden.
PUT • DELETE /api/app/ordenes/repuestos/{id}	Edita o quita un repuesto.

7.3 Configuración — AjustesController y SistemasController

Ruta base	Quién	Qué administra
/api/app/ajustes	Administrador, Sistemas	Categorías de activo de la organización y, dentro de cada una, sus estados y tipos de trabajo (con los estados de orden por los que pasa cada tipo).
/api/app/sistemas	Sistemas	Tipos de organización (módulos habilitados y textos), estados de orden, prioridades y estados de activo generales.

Cada operación de Ajustes y Sistemas devuelve la configuración completa ya actualizada, así la app refresca la pantalla con una sola respuesta.

8. Lógica de negocio detalle por detalle

8.1 Indicadores: cómo se “predicen” las fallas

La base no tiene columnas de MTBF ni predicciones: todo se **calcula** con el historial de órdenes de cada activo, en `Services/App/Indicadores.cs`.

Paso 1 — ¿Es falla o preventivo?

Una orden es **preventiva** si el nombre de su tipo de trabajo contiene palabras como *service, inspección, lubricación, revisión, aceite, filtro, control, calibración, limpieza, rutina*. Si no, se cuenta como **falla**.

Paso 2 — MTBF (tiempo medio entre fallas)

$\text{MTBF (días)} = \text{días desde la primera orden del activo} / \text{cantidad de fallas}$

Ejemplo: primera orden hace 120 días, 3 fallas → MTBF = 40 días

Paso 3 — Probabilidad de falla y nivel

$\text{probabilidad (\%)} = \text{días desde la última falla} / \text{MTBF} \times 100$ (tope 99 %)

Ejemplo: última falla hace 34 días, MTBF 40 → 85 % → nivel "riesgo"

Probabilidad	Nivel	Se muestra como
≥ 80 %	riesgo	En riesgo (alerta en el inicio)
≥ 60 %	revisar	Revisar
menor a 60 %	ok	OK

Paso 4 — Kilometraje (perfil taller)

En los talleres, al crear la orden se guarda el kilometraje al inicio de la descripción (Km ingreso: 128400). El próximo service se estima cada **10.000 km** desde el último service. Si el km actual lo supera, el nivel pasa a **riesgo** ("service vencido"); si faltan menos de 1.000 km, a **revisar**.

Paso 5 — Detenido y sugerencia

- Un activo está **detenido** si tiene una orden abierta ya iniciada, o si su estado contiene "detenido", "fuera de servicio", "parado" o "baja".
- Se genera una **sugerencia** de texto: "service vencido por 2.300 km: contactar al cliente", o "programar una revisión preventiva antes del 14/10".

Para defenderlo

"No usamos inteligencia artificial: es un modelo estadístico simple y explicable basado en el MTBF, un indicador estándar de mantenimiento. Lo calculamos en memoria a partir del historial, así cualquier orden nueva actualiza la predicción al instante, sin procesos batch ni columnas extra en la base."

8.2 La “foto” de la organización (Snapshot)

Dashboard, lista de activos, ficha y reportes necesitan lo mismo: los activos de la organización, sus órdenes y la salud de cada uno. `DatosOrganizacion.CargarAsync()` hace **dos consultas** (activos con sus

relaciones y órdenes con las suyas) y arma un Snapshot: diccionarios en memoria por id y la salud ya calculada de cada activo. Así cada pantalla se resuelve con 2 consultas, sin importar cuántos activos haya, en lugar de una consulta por activo.

8.3 Dashboard (pantalla de inicio)

- **Disponibilidad** = (activos totales – detenidos) / activos totales.
- **Alerta**: el activo más urgente (primero service vencido, después mayor probabilidad de falla).
- **Próximas**: 5 órdenes abiertas sin empezar, por fecha programada. **En curso**: 5 órdenes iniciadas.
- Turnos de hoy y services vencidos (taller).

8.4 Reportes

- Serie mensual de los últimos N meses: horas detenido por fallas, órdenes cerradas y costo (importe + repuestos).
- Variación contra el mes anterior, porcentaje de clientes recurrentes (con 2 o más órdenes).
- Activos críticos: los de más fallas; con 3 o más se sugiere "Evaluar reposición".
- Clientes a contactar: activos en riesgo con cliente, con la sugerencia para decirle.

8.5 Órdenes de trabajo

Creación

Se crea a nombre del usuario logueado, con fecha de creación del servidor. El estado inicial es el primer estado no final configurado para su tipo de trabajo. Se guardan los repuestos en `OrdenTrabajoDetalle` en la misma operación.

Cambio de estado — las fechas se completan solas

Si el nuevo estado es...	La API hace
Final (ej. Finalizada)	Pone FechaCierre = ahora (y FechaInicio si no tenía). Puede registrar importe y diagnóstico.
Intermedio (ej. En curso, Pausado)	Pone FechaInicio = ahora si no tenía, y borra FechaCierre (por si se reabre).
El estado inicial	No marca inicio (la orden sigue pendiente).

Flujo de estados por tipo de trabajo

La tabla `TipoTrabajoEstadoOrden` define por qué estados pasa cada tipo de trabajo (se configura en Ajustes). Si un tipo no tiene ninguno configurado, puede usar todos los estados.

Repuestos

Se pueden agregar, editar y quitar después de creada la orden. El costo total de la orden es el **importe** (mano de obra) más la suma de **cantidad** × **costo** de cada repuesto.

8.6 Lectura del código QR

El QR contiene el **Identificador** del activo (ej. una patente o un código interno). `BuscarPorCodigoAsync` es tolerante:

1. Acepta el código solo (C-02), una URL que termine en el código, o `activo:15` (por id).

2. Busca coincidencia exacta, sin distinguir mayúsculas.
3. Si no encuentra, reintentá ignorando espacios y guiones (AE 789 KL = AE789KL).
4. Siempre dentro de los activos de la organización (DatosOrganizacion).

8.7 Ajustes y Sistemas: reglas comunes

- **Sin duplicados:** no se puede crear dos veces el mismo nombre en el mismo ámbito (409 "Ya existe...").
- **Protección de lo usado:** cada elemento informa sus *usos* (activos u órdenes que lo referencian); si está en uso no se puede borrar.
- **Borrado de categoría:** borra también sus estados y tipos de trabajo, pero solo si nada los usa.
- **Módulos:** al quitar un módulo a un tipo de organización se borran sus textos asociados.
- Los textos (labels) cambian cómo la app nombra las cosas por rubro, por ejemplo LBL_TAB_ACTIVO = "Vehículos".

9. DTOs, validaciones y manejo de errores

9.1 ¿Qué es un DTO y por qué no devolver las entidades?

Un **DTO** (Data Transfer Object) es una clase que define exactamente qué datos entran o salen de un endpoint. Nunca se devuelven las entidades de la base directamente porque:

- Se filtrarían datos sensibles (ej. `ClaveHash` del usuario).
- La app recibe los datos ya combinados y calculados (ej. `ActivoResumenDto` trae el nombre del cliente y la salud).
- Se puede cambiar la base sin romper la app, y viceversa.

9.2 Validación automática

Los DTOs de entrada tienen atributos (*DataAnnotations*). Gracias a `[ApiController]`, si el JSON no cumple, ASP.NET responde **400** automáticamente, con el detalle por campo, antes de llegar al servicio:

DTOS/App/AppDtos.cs

```
public sealed class ActivoGuardarDto
{
    [Required]
    [StringLength(150)]
    public string Nombre { get; set; } = string.Empty;    // obligatorio, máx. 150

    [StringLength(100)]
    public string? Identificador { get; set; }
    ...
}
```

9.3 Errores: siempre la misma forma

Los servicios, ante una regla violada, lanzan una excepción. `GlobalExceptionHandler` la atrapa en un solo lugar y responde en formato estándar **ProblemDetails** (`title`, `detail`, `status`), que la app muestra al usuario:

Excepción	HTTP	Cuándo
<code>ConflictException</code>	409	Regla de negocio: nombre duplicado, borrar algo en uso, categoría de otra organización...
<code>UnauthorizedAccessException</code>	401	El token no identifica a un usuario.
<code>DbUpdateException</code>	409	La base rechazó el cambio (restricción de clave foránea).
Cualquier otra	500	Error inesperado: se registra en el log y al usuario se le da un mensaje genérico (no se filtran detalles internos).

Ejemplo de respuesta de error

```
{
  "title": "Conflicto",
  "status": 409,
  "detail": "No se puede eliminar: tiene 1 orden(es) de trabajo en su historial."
}
```

Código	Significado en esta API
200 / 201 / 204	OK / creado / OK sin contenido (ej. al eliminar).
400	Datos inválidos (falta un campo obligatorio, texto muy largo...).
401	Sin token, token vencido o inválido.
403	Autenticado pero sin el rol necesario (Ajustes, Sistemas).
404	No existe o no pertenece a tu organización.
409	Conflicto con una regla de negocio.

10. Configuración y puesta en marcha

10.1 Cómo levantarla

```
cd Mantenia.Api
dotnet restore
dotnet run --launch-profile http
# → http://localhost:5080/scalar/v1 (interfaz para probar los endpoints)
```

10.2 appsettings

Clave	Para qué
ConnectionStrings:ConexionSql	Cadena de conexión a SQL Server.
Jwt:Key / Issuer / Audience / ExpiracionHoras	Firma y datos del token (24 hs por defecto). La API no arranca si la clave tiene menos de 32 caracteres.
Auth:PermitirRegistro	Habilita o no el alta pública de usuarios.
Cors:Origenes	Qué direcciones (frontends) pueden llamar a la API desde el navegador.

Buena práctica

En producción, la cadena de conexión y `Jwt:Key` no se escriben en el archivo: se cargan por variables de entorno o `dotnet user-secrets`, para no subir secretos al repositorio.

10.3 Scripts SQL

Script	Qué hace
Rol_Sistemas.sql	Crea el rol "Sistemas" y se lo asigna al usuario del equipo de sistemas.
EstadoActivo_Programado.sql	Crea el estado general "Programado" (disponible para todas las categorías).

Ambos se pueden ejecutar más de una vez sin duplicar datos (verifican antes de insertar).

10.4 Crear un usuario

1. Con `Auth:PermitirRegistro = true`, abrir Scalar y ejecutar POST `/api/auth/registro` con nombre, email, clave (mín. 8), organización y rol.
2. Probar el login con ese usuario.
3. Poner `PermitirRegistro` en `false` en producción.

11. Preguntas frecuentes para la defensa

¿Por qué una API y no que la app lea la base directamente?

Porque la base no puede quedar expuesta a internet ni confiar en el celular del usuario. La API es el único punto que decide quién puede ver y hacer qué, valida los datos y aplica las reglas en un solo lugar.

¿Cómo evitan que una organización vea datos de otra?

Todas las consultas de activos y órdenes parten de un único método, `ActivosQuery()`, que filtra por la organización del usuario leída de la base. Lo de otra organización responde 404 y no se puede referenciar al crear o editar (409).

¿Por qué el rol y la organización se leen de la base y no del token?

Porque el token dura 24 horas. Si se lee de la base, un cambio de rol, de organización o una baja aplican en el acto. El costo es una consulta muy liviana por pedido.

¿Cómo se guardan las contraseñas?

Con hash PBKDF2 + HMAC-SHA512, sal aleatoria y 100.000 iteraciones (`PasswordHasher` de ASP.NET Core Identity). Nunca en texto plano y nunca se devuelven.

¿Por qué no usaron migraciones de Entity Framework?

Porque la base ya existía con su estructura definida y la API no debe alterarla. Las clases se escribieron para calzar con las tablas (Database First); los datos iniciales se cargan con scripts SQL idempotentes.

¿Por qué los controllers no tienen lógica?

Para respetar la separación de responsabilidades: el controller traduce HTTP y el servicio aplica las reglas. Así la lógica se puede reutilizar y probar sin HTTP.

¿Cómo funciona la predicción de fallas?

Con el MTBF (tiempo medio entre fallas) calculado del historial: días observados / fallas. La probabilidad compara los días desde la última falla contra el MTBF; $\geq 80\%$ es riesgo. En talleres se suma el control de kilometraje (service cada 10.000 km).

¿No es lento calcular todo en cada pedido?

Se hacen solo 2 consultas por pantalla (Snapshot) y el cálculo es en memoria, del orden de milisegundos para cientos de activos. Si creciera mucho, el siguiente paso sería cachear el Snapshot por organización.

¿Qué pasa si hay un error inesperado?

`GlobalExceptionHandler` lo atrapa, lo registra en el log y devuelve un 500 con un mensaje genérico, sin exponer detalles internos.

¿Por qué se eliminó el CRUD genérico de las 27 tablas?

Porque la app no lo usaba y era una puerta abierta: permitía modificar cualquier tabla sin filtro de organización. Se dejó solo lo que la app necesita, con endpoints pensados por pantalla y con sus reglas.

¿Qué mejorarían a futuro?

Agregar `IdOrganizacion` a `Activo` y `Ubicacion` (hoy la pertenencia se deduce por categoría o cliente), pruebas automatizadas, cache del Snapshot, y órdenes programadas recurrentes (ej. cambio de aceite cada año) para el estado "Programado".

12. Glosario

Término	Significado
API / endpoint	Programa que expone funciones por HTTP. Cada URL + método (GET, POST...) es un endpoint.
Controller	Clase que agrupa endpoints y traduce HTTP ↔ llamadas a servicios.
Service	Clase con la lógica de negocio.
DTO	Objeto que define los datos que entran o salen de un endpoint.
Entity	Clase que representa una tabla de la base.
ORM / EF Core	Herramienta que traduce objetos y consultas C# a SQL.
DbContext	La "sesión" con la base: expone las tablas y guarda los cambios.
LINQ	Sintaxis de C# para consultar colecciones o la base (Where, Select, OrderBy...).
JWT	Token firmado que identifica al usuario en cada pedido.
Claim	Cada dato que lleva el token (id de usuario, email...).
Hash / PBKDF2	Transformación irreversible de la clave, con sal e iteraciones.
Middleware	Paso por el que atraviesa cada pedido (autenticación, CORS, errores...).
Política de autorización	Regla que decide si un usuario autenticado puede usar un endpoint.
Multi-tenant	Varias organizaciones comparten el sistema y la base, con los datos aislados.
Inyección de dependencias	El framework crea y entrega las dependencias de cada clase por su constructor.
ProblemDetails	Formato estándar (RFC 9457) para devolver errores en JSON.
MTBF	Mean Time Between Failures: tiempo medio entre fallas.
CORS	Mecanismo del navegador que limita qué sitios pueden llamar a la API.
Scalar / OpenAPI	Documentación de la API generada automáticamente y web para probarla.